Documento de Arquitetura de Software (DAS) – Aplicação 'Risco em Perspectiva'

A documentação formal da arquitetura da aplicação 'Risco em Perspectiva' estabelece o alinhamento técnico e estratégico entre os times de engenharia de software, compliance, segurança da informação e analistas de risco corporativo. Este documento especifica as diretrizes de projeto, estruturas de dados, fluxos de integração e modelos de análise, servindo como guia definitivo para a evolução, auditoria e operação continuada do sistema em ambientes restritos.

1. Visão Geral e Propósito do Sistema

O 'Risco em Perspectiva' é uma plataforma especializada na gestão, classificação e análise de riscos operacionais e corporativos. O sistema foi projetado para unificar a avaliação qualitativa matricial com simulações quantitativas estocásticas de impacto financeiro, operando de forma totalmente isolada.

1.1. Escopo e Objetivos do Sistema

O objetivo primordial do sistema é permitir o cadastramento estruturado de riscos, a avaliação detalhada de dimensões operacionais, o cálculo automatizado de exposição financeira e a geração de pareceres normativos suportados por Inteligência Artificial local.

Escopo Coberto	Fora de Escopo
Avaliação Matricial 4Ps: Análise qualitativa cobrindo as dimensões `Pessoas`, `Processos`, `Produtos` e `Políticas governamentais` (escala 1 a 5).	**Estimativa Monetária Direta via Matriz 4Ps:** A análise matricial reativa dos 4Ps não calcula valores financeiros nem perdas diretas em dinheiro no painel principal.
Quantificação Financeira (FAIR): Simulação de perdas financeiras em R$/ano baseada no framework OpenFAIR via Monte Carlo (módulo estendido).	**Conexão com APIs de IA em Nuvem:** Integração ou envio de telemetria/dados para serviços externos de IA (ex.: OpenAI, Anthropic).

Justificativas por IA Local: Geração e sugestão de textos descritivos e recomendações executadas exclusivamente via motor local (Ollama).	**Persistência Remota de Dados:** Dependência de bancos de dados gerenciados em nuvem ou servidores remotos externos.

1.2. Abordagem Híbrida de Análise: Matriz Qualitativa 4Ps e Quantificação Financeira FAIR

O sistema adota uma arquitetura conceitual de dupla camada analítica:

1. **Camada Qualitativa Matricial (4Ps):** Avalia o risco sob quatro perspectivas independentes e complementares: `Pessoas`, `Processos`, `Produtos` e `Políticas governamentais`. Cada dimensão é pontuada individualmente nos quesitos **Severidade** (1 a 5) e **Probabilidade** (1 a 5), resultando em uma nota de 1 a 25.
2. **Camada Quantificável Estocástica (OpenFAIR):** Emprega a biblioteca `pyfair` para executar simulações de Monte Carlo. Essa camada converte os níveis discretos de probabilidade e severidade em distribuições de frequência de eventos e magnitude de perdas financeiras em moeda local (R$/ano).

Essa abordagem híbrida permite à diretoria executiva visualizar simultaneamente onde as falhas operacionais estão concentradas (através do painel 4Ps) e qual é o impacto financeiro provável e extremo (P95/VaR) associado à carteira de riscos.

1.3. Arquitetura Air-Gapped e Execução de IA Local

Para atender aos rigorosos requisitos de privacidade, confidencialidade e isolamento de dados corporativos, o sistema opera sob o conceito *air-gapped*. O processamento de linguagem natural para geração de justificativas dimensionais e sugestões de mitigação é realizado localmente por meio do servidor Ollama, utilizando modelos da família `deepseek-r1` (com suporte automático a tags como `7b`, `14b`, entre outras).

A segurança do fluxo de dados é garantida por uma validação estrita no código da aplicação (`_validate_local_url`), que restringe qualquer chamada de API de IA ao endereço de *loopback* local (`127.0.0.1`, `localhost`, `::1`). Redirecionamentos para proxies do sistema operacional são desativados nativamente, assegurando vazamento zero de informações sensíveis para redes externas.

A compreensão dos objetivos estratégicos e das restrições operacionais do sistema fundamenta a topologia técnica e a organização dos componentes da aplicação.

2. Visão da Arquitetura de Software e Componentes

A topologia da aplicação organiza-se em uma estrutura modular desacoplada em Single Page Application (SPA) para a camada de apresentação e uma API REST local para a camada de aplicação e serviços analíticos, garantindo manutenibilidade, extensibilidade e clara separação de responsabilidades.

2.1. Arquitetura Geral do Sistema e Camadas

O sistema é construído como um monólito leve com frontend e backend bem delimitados. A estrutura de diretórios e arquivos centrais reflete essa divisão:

```
/ (Raiz do Projeto)
├── frontend/               # Single Page Application em Vue 3 (Interface do Usuário)
│   └── src/Components/     # Componentes visuais e painéis interativos
├── api.py                  # Roteamento FastAPI, validação Pydantic e controle de concorrência
├── analysis_service.py     # Motor FAIR, simulação Monte Carlo estocástica e persistência central
├── rating_analysis.py      # Processamento matricial 4Ps, histórico de notas e relatórios reativos
├── local_ai.py             # Cliente HTTP do Ollama local, engenharia de prompts e sanitização
├── analise_riscos.py       # Módulo de scripts analíticos autônomos e modelagem PyFAIR CLI
└── DataBase.db             # Banco de dados SQLite local
```

2.2. Frontend (Vue 3 SPA)

Desenvolvido sobre o ecossistema Node.js 20+ e **npm**, o frontend em Vue 3 é responsável por:

- Consumir a API REST exposta pelo backend Python.
- Gerenciar o estado da interface em tempo real durante a edição de riscos e execução de simulações.
- Apresentar visualizações dinâmicas, incluindo a Matriz de Risco 5x5 interativa (filtrável por dimensão), gráficos de distribuição de notas (baixas, médias e altas), históricos de pareceres da carteira e formulários de cadastro de evidências e justificativas.

2.3. Backend API (FastAPI & Application Layer) e Padrão Arquitetural de Duplo Motor

O backend é suportado pelo framework **FastAPI** rodando em **Python 3.13+**. O sistema implementa o padrão de **Arquitetura de Duplo Motor (*Dual-Engine Architecture*)**:

1. **Motor Matricial Qualitativo Reativo (`rating_analysis.py`):** Integrado diretamente ao fluxo HTTP principal do frontend. O endpoint `POST /api/analysis` executa a função `run_rating_analysis()`, que processa instantaneamente as notas dos 4Ps, consolida os pareceres categóricos e gera os relatórios operacionais

(`relatorio_4ps.html` e `relatorio_4ps.xlsx`). Isso permite feedback imediato ao usuário sem latência de processamento estocástico.

2. **Motor Quantitativo Estocástico Avançado (`analysis_service.py` / `analise_riscos.py`):** Atua como um motor de cálculo intensivo desacoplado. Executa simulações de Monte Carlo via OpenFAIR (10.000 iterações por risco e dimensão), gerando estimativas monetárias e relatórios de exposição financeira (`relatorio_fair.html` e `relatorio_fair.xlsx`).

O evento de ciclo de vida (`lifespan`) do FastAPI invoca a função `initialize_database()`, garantindo a criação e migração do esquema SQLite antes de aceitar tráfego.

2.4. Módulo de Serviços e Regras de Negócio (`analysis_service.py` e `rating_analysis.py`)

A lógica de negócios é dividida estrategicamente entre os dois módulos de serviço:

- `rating_analysis.py`: Focado na análise matricial 4Ps em tempo real. Avalia severidade e probabilidade, compõe o histórico de classificações em `analysis_history` (salvando métricas em JSON sob a chave `rating_metrics`), atualiza a tabela `dimension_results_matrix` e compila os relatórios `relatorio_4ps.html` e `relatorio_4ps.xlsx`.
- `analysis_service.py`: Concentra a persistência CRUD de riscos (`riscos` e `risk_dimensions`), a inicialização do banco, o gerenciamento de *benchmarks* e a execução da simulação estocástica FAIR. Escreve os resultados nas tabelas `resultados_fair` e `dimension_results_fair`, além de armazenar artefatos estocásticos em disk (`relatorio_fair.html` e `relatorio_fair.xlsx`).

2.5. Integração com IA Local (`local_ai.py` e Pipeline de Tolerância a Falhas)

O módulo `local_ai.py` abstrai a comunicação REST HTTP com o serviço **Ollama**:

- **Configuração:** Lê `OLLAMA_BASE_URL` (padrão `http://127.0.0.1:11434`) e `OLLAMA_MODEL` (padrão `deepseek-r1`).
- **Seleção Dinâmica de Modelos:** A função `_select_installed_model` inspeciona as tags retornadas por `/api/tags` e seleciona automaticamente a versão instalada correspondente à família (ex.: `deepseek-r1:14b`).
- **Timeouts:** Limita em 3 segundos a verificação de integridade e 90 segundos a geração do texto (`/api/generate`).
- **Pipeline de Sanitização e Tolerância a Falhas Multi-estágio:** Para mitigar o não-determinismo de LLMs locais:

1. Força a saída em formato estruturado via `"format": "json"` nas chamadas de recomendação.
2. Remoção de blocos de raciocínio delimitados pela tag `</think>` (padrão dos modelos DeepSeek R1).
3. Extração de payloads JSON via expressão regular (`re.search(r"\{.*\}", response, re.DOTALL)`), prevenindo erros de sintaxe decorrentes de texto introdutório.
4. Re-indexação automática e validação de limites para escalas de prioridade (1 a 5) com base na ordem dos itens retornados.
5. Truncamento estrito de caracteres (`action[:300]` e `justification[:700]` para sugestões; `1200` caracteres para justificativas) e temperatura de amostragem fixada em `0.2`.

2.6. Camada de Persistência (SQLite Engine)

A persistência de dados utiliza o motor **SQLite**, gerando ou consumindo o arquivo local `DataBase.db`. O caminho do banco pode ser customizado via variável de ambiente `RISK_DB_PATH`. Essa escolha garante portabilidade total da aplicação e facilidade de implantação sem a necessidade de infraestrutura de banco de dados relacional externa.

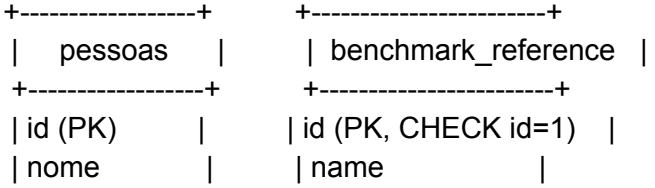
A interação fluida entre os componentes da aplicação depende de um modelo de dados coerente para estruturar as informações operacionais e quantitativas do sistema.

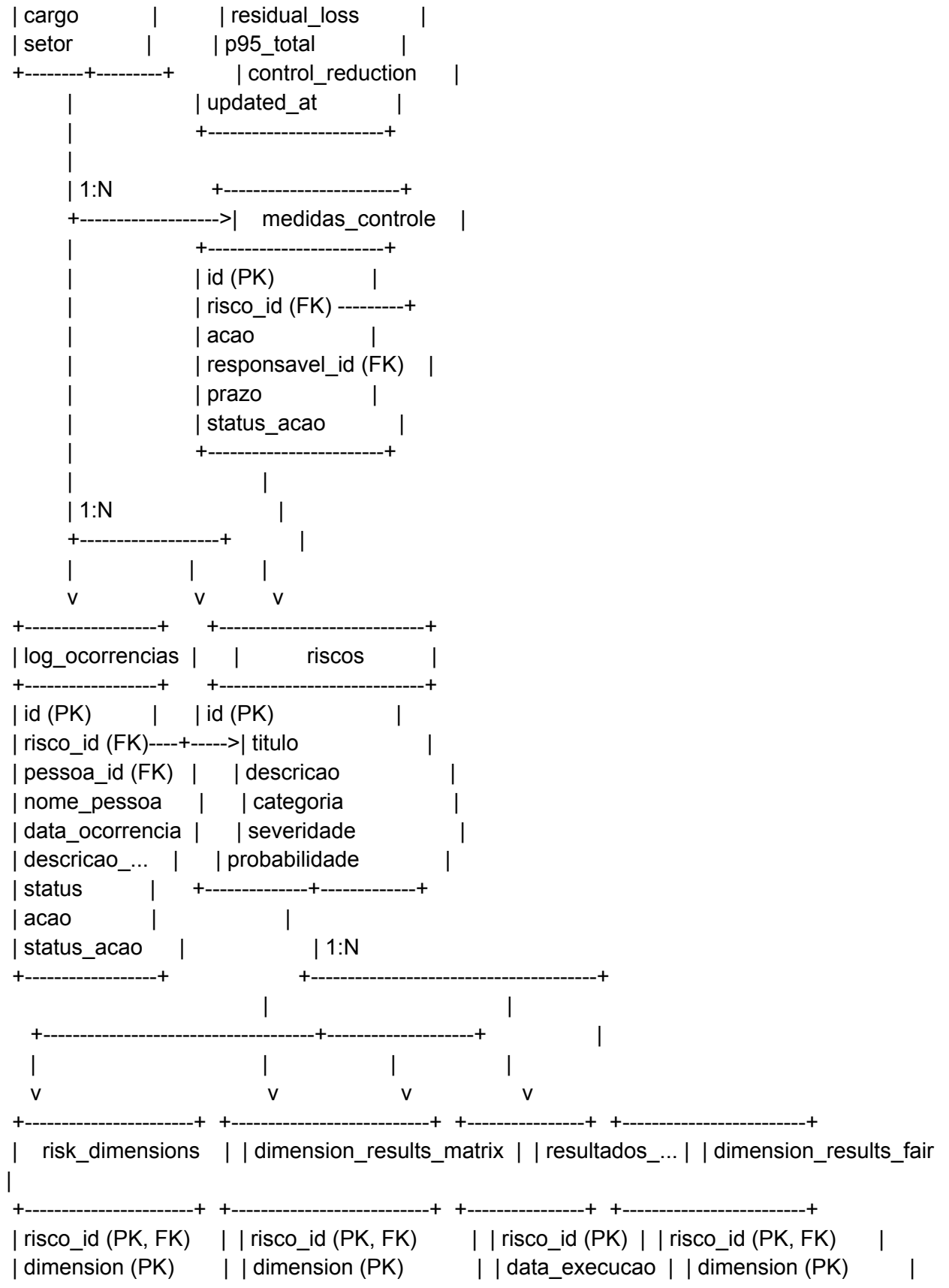
3. Visão de Dados e Modelagem do Banco de Dados (SQLite)

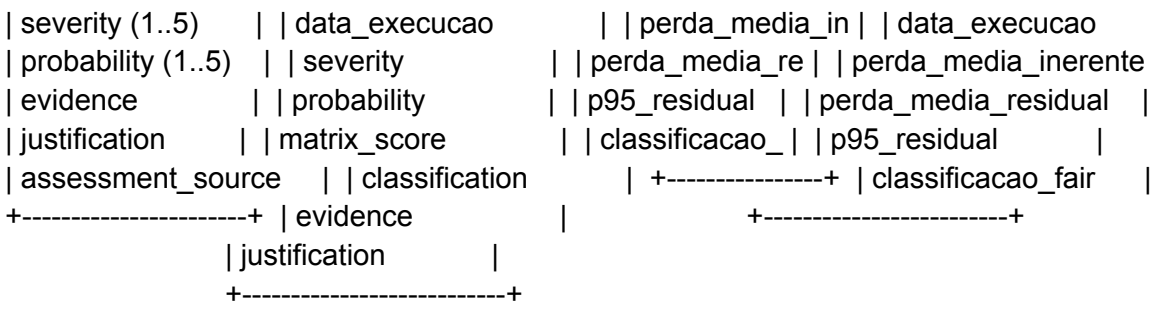
O modelo relacional em SQLite foi projetado para acomodar o cadastro operacional de riscos, o registro de equipes, a definição granular das dimensões 4Ps, o acompanhamento de planos de ação, o histórico de notas qualitativas e as métricas de simulações estatísticas.

3.1. Esquema de Entidades e Relacionamentos

A arquitetura do banco baseia-se em uma entidade central (`riscos`), à qual conectam-se as dimensões operacionais (`risk_dimensions`), os históricos de execuções, os logs de ocorrências e os controles de mitigação. As tabelas secundárias possuem restrições de integridade referencial (*Foreign Keys*), com suporte a exclusões em cascata (`ON DELETE CASCADE`) na tabela `risk_dimensions` para manter a consistência do banco durante operações de descarte de registros.




| cargo | | residual_loss |
| setor | | p95_total |
+--------+---------+ | control_reduction |
| | updated_at |
| +----------+
| 1:N +----------+
+----------->| medidas_controle |
| +----------+
| | id (PK) |
| | risco_id (FK) ---------+
	acao
	responsavel_id (FK)
	prazo
	status_acao
+----------+
1:N
+----------+
v v v
+----------+ +----------+
| log_ocorrencias | | riscos |
+----------+ +----------+
id (PK)		id (PK)
risco_id (FK)----+----->	titulo
pessoa_id (FK)		descricao
nome_pessoa		categoria
data_ocorrencia		severidade
descricao_...		probabilidade
status	+----------+----------+
acao	
status_acao		1:N
+----------+ +----------+
+----------+----------+
v v v v
+----------+ +----------+ +----------+ +----------+
| risk_dimensions | | dimension_results_matrix | | resultados_... | | dimension_results_fair
|
+----------+ +----------+ +----------+ +----------+
| risco_id (PK, FK) | | risco_id (PK, FK) | | risco_id (PK) | | risco_id (PK, FK) |
| dimension (PK) | | dimension (PK) | | data_execucao | | dimension (PK) |



3.2. Estrutura Detalhada e Restrições das Tabelas do Sistema

- **`pessoas`**: Cadastro de colaboradores e responsáveis técnicos.
  - `id`: `INTEGER PRIMARY KEY AUTOINCREMENT`
  - `nome`: `TEXT NOT NULL`
  - `cargo`: `TEXT`
  - `setor`: `TEXT`
- **`riscos`**: Tabela principal de cadastro de eventos de risco.
  - `id`: `INTEGER PRIMARY KEY AUTOINCREMENT`
  - `titulo`: `TEXT NOT NULL`
  - `descricao`: `TEXT`
  - `categoria`: `TEXT`
  - `severidade`: `INTEGER`
  - `probabilidade`: `INTEGER`
- **`risk_dimensions`**: Registro detalhado dos 4Ps por risco.
  - *Chave Primária Composta:* `PRIMARY KEY (risco_id, dimension)`
  - `risco_id`: `INTEGER NOT NULL`, `FOREIGN KEY REFERENCES riscos(id) ON DELETE CASCADE`
  - `dimension`: `TEXT NOT NULL` (`Pessoas`, `Processos`, `Produtos`, `Políticas governamentais`)
  - `severity`: `INTEGER NOT NULL`, `CHECK (severity BETWEEN 1 AND 5)`
  - `probability`: `INTEGER NOT NULL`, `CHECK (probability BETWEEN 1 AND 5)`
  - `evidence`: `TEXT NOT NULL DEFAULT ''`
  - `justification`: `TEXT NOT NULL DEFAULT ''`
  - `assessment_source`: `TEXT NOT NULL DEFAULT 'assessed'` (`assessed` ou `inherited`)
- **`dimension_results_matrix`**: Armazena os resultados da análise matricial qualitativa.
  - *Chave Primária Composta:* `PRIMARY KEY (risco_id, dimension)`

    - risco_id: INTEGER NOT NULL, FOREIGN KEY REFERENCES riscos(id)
    - dimension: TEXT NOT NULL
    - data_execucao: TEXT NOT NULL
    - severity: INTEGER NOT NULL
    - probability: INTEGER NOT NULL
    - matrix_score: INTEGER NOT NULL (Nota de 1 a 25)
    - classification: TEXT NOT NULL (Baixo, Médio, Alto)
    - evidence: TEXT NOT NULL
    - justification: TEXT NOT NULL
- **resultados_fair**: Consolidação financeira global por risco pós-simulação FAIR.
    - risco_id: INTEGER PRIMARY KEY, FOREIGN KEY REFERENCES riscos(id)
    - data_execucao: TEXT
    - perda_media_inerente: REAL
    - perda_media_residual: REAL
    - p95_residual: REAL (Value at Risk 95%)
    - classificacao_fair: TEXT (Baixo, Médio, Alto)
- **dimension_results_fair**: Resultados FAIR detalhados por dimensão individual.
    - *Chave Primária Composta:* PRIMARY KEY (risco_id, dimension)
    - risco_id: INTEGER NOT NULL, FOREIGN KEY REFERENCES riscos(id)
    - dimension: TEXT NOT NULL
    - data_execucao: TEXT NOT NULL
    - perda_media_inerente: REAL NOT NULL
    - perda_media_residual: REAL NOT NULL
    - p95_residual: REAL NOT NULL
    - classificacao_fair: TEXT NOT NULL
- **analises_risco**: Registro histórico simples das análises.
    - id: INTEGER PRIMARY KEY AUTOINCREMENT
    - risco_id: INTEGER, FOREIGN KEY REFERENCES riscos(id)
    - pessoa_id: INTEGER, FOREIGN KEY REFERENCES pessoas(id)
    - data_analise: TEXT NOT NULL
    - nivel_risco_calculado: INTEGER
    - status: TEXT
- **medidas_controle**: Ações de mitigação atreladas aos riscos cadastrados.
    - id: INTEGER PRIMARY KEY AUTOINCREMENT
    - risco_id: INTEGER, FOREIGN KEY REFERENCES riscos(id)
    - acao: TEXT NOT NULL
    - responsavel_id: INTEGER, FOREIGN KEY REFERENCES pessoas(id)

    - `prazo`: `TEXT`
    - `status_acao`: `TEXT` (`Concluída`, `Em andamento`, `Pendente`)
- **`analysis_history`**: Snapshots de execuções históricas e métricas da carteira.
    - `id`: `INTEGER PRIMARY KEY AUTOINCREMENT`
    - `analyzed_at`: `TEXT NOT NULL`
    - `inherent_loss`: `REAL NOT NULL`
    - `residual_loss`: `REAL NOT NULL`
    - `p95_total`: `REAL NOT NULL`
    - `control_reduction`: `REAL NOT NULL`
    - `risk_count`: `INTEGER NOT NULL`
    - `simulations_per_risk`: `INTEGER NOT NULL`
    - `dimension_metrics`: `TEXT NOT NULL DEFAULT '{}'` (Payload JSON de métricas dimensionais)
    - `rating_metrics`: `TEXT NOT NULL DEFAULT '{}'` (Payload JSON de resultados da matriz)
- **`benchmark_reference`**: Parâmetros de referência externa da organização.
    - `id`: `INTEGER PRIMARY KEY`, `CHECK (id = 1)` (Garante registro único)
    - `name`: `TEXT NOT NULL`
    - `residual_loss`: `REAL`
    - `p95_total`: `REAL`
    - `control_reduction`: `REAL`
    - `updated_at`: `TEXT NOT NULL`
- **`log_ocorrencias`**: Registro de histórico de incidentes operacionais reais.
    - `id`: `INTEGER PRIMARY KEY AUTOINCREMENT`
    - `risco_id`: `INTEGER`, `FOREIGN KEY REFERENCES riscos(id)`
    - `pessoa_id`: `INTEGER`, `FOREIGN KEY REFERENCES pessoas(id)`
    - `nome_pessoa`: `TEXT`
    - `data_ocorrencia`: `TEXT`
    - `descricao_ocorrencia`: `TEXT`
    - `status`: `TEXT`
    - `acao`: `TEXT`
    - `status_acao`: `TEXT`

3.3. Política de Migração, Mapeamento Dinâmico e Notas Herdadas

A função `initialize_database` garante a evolução do banco de dados sem perda de dados existentes. Durante a inicialização, o sistema inspeciona as tabelas via instrução `PRAGMA table_info(...)`. Se colunas como `dimension_metrics`, `rating_metrics`, `evidence` ou `justification` não forem localizadas, a aplicação executa comandos dinâmicos `ALTER TABLE` para atualizar o esquema.

Para manter a compatibilidade com registros cadastrados em versões anteriores que não possuíam avaliações divididas pelos 4Ps, a rotina de inicialização cria automaticamente quatro entradas em `risk_dimensions` para cada risco legado, clonando a severidade e a probabilidade gerais e marcando o campo `assessment_source` com o valor `'inherited'`.

Impacto Funcional: A presença de qualquer avaliação marcada como `inherited` ou sem justificativa preenchida altera a propriedade do parecer de risco para `provisorio: true`. O sistema emite um alerta explícito informando que o parecer final está "pendente de revisão", exigindo a edição e validação manual pelo analista de risco.

Os dados armazenados e migrados estruturadamente servem de insumo direto para a execução das regras de negócio e dos modelos matemáticos do sistema.

4. Regras de Negócio e Modelagem Matemática

O motor analítico do sistema combina formulações determinísticas para a avaliação matricial qualitativa com métodos estocásticos baseados em Monte Carlo para a quantificação do risco financeiro.

4.1. Avaliação Matricial 4Ps e Critério de Parecer Final

A pontuação de cada uma das quatro dimensões (`Pessoas`, `Processos`, `Produtos`, `Políticas governamentais`) é calculada de forma independente:

\text{Nota da Dimensão} = \text{Severidade (1..5)} \times \text{Probabilidade (1..5)}

A nota resultante (variando de 1 a 25) é enquadrada nas seguintes faixas classificatórias:

- **Baixo:** Nota de 1 a 7
- **Médio:** Nota de 8 a 14
- **Alto:** Nota de 15 a 25

Algoritmo do Parecer Final do Risco: O painel de avaliação emite um parecer final categórico para cada risco:

- **"Favorável":** O parecer é considerado *Favorável* **apenas e tão somente se** todas as quatro dimensões (Pessoas, Processos, Produtos e Políticas governamentais) obtiverem classificação na faixa **Baixo** (1 a 7).
- **"Não favorável":** Caso **qualquer uma** das dimensões atinja a faixa **Médio** (8 a 14) ou **Alto** (15 a 25), o risco é automaticamente classificado como *Não favorável*.

Se o risco possuir notas herdadas (`assessment_source = 'inherited'`) ou justificativas dimensionais em branco, o parecer é sinalizado como **provisório**, requerendo revisão obrigatória dos cadastros.

4.2. Quantificação Financeira de Risco via PyFAIR (Simulação Monte Carlo)

No módulo quantitativo estendido (`analysis_service.py`), o motor estocástico mapeia as notas qualitativas de probabilidade e severidade para parâmetros discretos da distribuição PERT (Mínimo, Mais Provável, Máximo), utilizados pelo framework `pyfair`.

Mapeamento de Frequência de Eventos de Perda (Probabilidade 1 a 5):

Nota de Probabilidade	Frequência Anual (Mínimo, Mais Provável, Máximo)
1	0.01, 0.03, 0.10 eventos/ano
2	0.05, 0.15, 0.40 eventos/ano
3	0.20, 0.60, 1.50 eventos/ano
4	0.80, 2.00, 4.00 eventos/ano
5	2.00, 5.00, 12.00 eventos/ano

Mapeamento de Magnitude da Perda Financeira (Severidade 1 a 5):

Nota de Severidade	Perda por Evento em R$ (Mínimo, Mais Provável, Máximo)
1	R 1.000 / R 5.000 / R$ 20.000
2	R 5.000 / R 25.000 / R$ 100.000

3	R 30.000 / R 120.000 / R$ 500.000
4	R 150.000 / R 600.000 / R$ 2.500.000
5	R 500.000 / R 2.500.000 / R$ 10.000.000

O sistema executa **10.000 simulações por risco/dimensão** (`SIMULATIONS_PER_RISK = 10000`). A partir das iterações, são calculadas a **Perda Média Inerente**, a **Perda Média Residual** e o **P95 Residual** (Value at Risk correspondente ao percentil 95 de perda nas simulações).

Limiares de Exposição Financeira FAIR:

- **Baixo:** Perda média residual < R$ 50.000 / ano
- **Médio:** Perda média residual entre R 50.000 e R 200.000 / ano
- **Alto:** Perda média residual \ge R$ 200.000 / ano

4.3. Algoritmo de Redução de Frequência por Medidas de Controle

As ações cadastradas na tabela `medidas_controle` reduzem a frequência esperada de ocorrência do risco em sua avaliação residual. Cada medida contribui com um fator percentual de acordo com o seu *status*:

- Ação **"Concluída"**: soma 0,30 (30%) de redução.
- Ação **"Em andamento"**: soma 0,10 (10%) de redução.
- Ação **"Pendente"**: soma 0,00 (0%) de redução.

A soma dos percentuais é limitada pela trava matemática de redução máxima acumulada:

\text{Fator de Redução Acumulado} = \min\left(\sum \text{Redução por Status}, 0.70\right)

Na construção do modelo FAIR residual, os parâmetros da distribuição PERT de frequência de perda são multiplicados pelo fator (1 - \text{Fator de Redução Acumulado}), simulando o efeito protetivo dos controles aplicados.

A formalização matemática das regras de negócio é exposta para o ambiente cliente por meio de contratos de API padronizados.

5. Especificação de Endpoints da API REST

A comunicação entre a interface Vue 3 e o servidor FastAPI ocorre por meio de uma API RESTful formatada em JSON, utilizando códigos de status HTTP normatizados para indicar o resultado das operações.

5.1. Módulo de Gestão de Riscos (CRUD Operacional)

- `GET /api/risks`
  - **Descrição:** Retorna a lista de todos os riscos cadastrados e o detalhamento de suas avaliações nas 4 dimensões.
  - **Resposta:** `200 OK` [Lista de Riscos]
- `POST /api/risks`
  - **Descrição:** Cadastra um novo risco e exige o envio das avaliações das 4 dimensões (`Pessoas`, `Processos`, `Produtos`, `Políticas governamentais`).
  - **Resposta:** `201 Created`
- `PUT /api/risks/{risk_id}`
  - **Descrição:** Atualiza os dados cadastrais e as pontuações dimensionais de um risco existente.
  - **Resposta:** `200 OK` ou `404 Not Found`
- `DELETE /api/risks/{risk_id}`
  - **Descrição:** Remove um risco do banco de dados e apaga em cascata seus registros vinculados em tabelas associadas.
  - **Resposta:** `200 OK {"deleted": true, "risk_id": risk_id}` ou `404 Not Found`
- `PUT /api/risks/{risk_id}/dimensions`
  - **Descrição:** Atualiza exclusivamente as pontuações, evidências e justificativas das 4 dimensões de um risco específico.
  - **Resposta:** `200 OK` ou `404 Not Found`

5.2. Módulo de Execução de Análises e Monitoramento

- `POST /api/analysis`
  - **Descrição:** Dispara a análise matricial reativa dos 4Ps chamando `run_rating_analysis()`. Atualiza a tabela `dimension_results_matrix`, grava a métrica no histórico (`analysis_history`) e gera os relatórios em disco (`relatorio_4ps.html` / `relatorio_4ps.xlsx`).
  - **Resposta:** `200 OK` ou `409 Conflict` (se outra análise estiver em curso).
- `GET /api/analysis/progress`
  - **Descrição:** Consulta o estado de execução da análise em tempo real.
  - **Resposta:** `200 OK {"status": "running"|"completed"|"failed", "percent": 0..100, "message": "..."}`
- `GET /api/analysis/history`

    - **Descrição:** Recupera os últimos 20 históricos de execuções registradas via `get_rating_history()`.
    - **Resposta:** `200 OK` [Lista de Execuções]

5.3. Módulo de Inteligência Artificial Local

- `GET /api/ai/status`
    - **Descrição:** Checa a disponibilidade do servidor Ollama local e confirma se o modelo configurado (ex.: `deepseek-r1`) está carregado.
    - **Resposta:** `200 OK {"available": true|false, "model": "...", "message": "..."}`
- `POST /api/ai/dimension-justification`
    - **Descrição:** Envia o contexto dimensional de um risco (título, categoria, dimensão, severidade, probabilidade e evidências) para o Ollama local e retorna uma proposta de justificativa sintética (1 a 3 frases) com temperatura 0.2.
    - **Resposta:** `200 OK {"justification": "..."}`, `422 Unprocessable Entity` (dados inválidos) ou `503 Service Unavailable` (Ollama inacessível).
- `POST /api/ai/resolution-suggestions`
    - **Descrição:** Envia a avaliação completa dos 4Ps de um risco e solicita de 3 a 5 recomendações práticas de prevenção priorizadas.
    - **Resposta:** `200 OK {"suggestions": [{"action": "...", "priority": 1..5, "justification": "..."}, ...]}`

5.4. Módulo de Exportação de Relatórios e Utilitários

- `GET /api/health`
    - **Descrição:** Ponto de checagem básica de operacionalidade do servidor web.
    - **Resposta:** `200 OK {"status": "ok"}`
- `GET /api/reports/html`
    - **Descrição:** Baixa o relatório analítico compilado mais recente da matriz 4Ps (`relatorio_4ps.html`).
    - **Cabeçalho de Resposta:** `Content-Disposition: attachment; filename="relatorio_4ps.html"`
    - **Resposta:** `200 OK` (`text/html`) ou `409 Conflict` (caso nenhuma análise tenha sido executada).
- `GET /api/reports/excel`
    - **Descrição:** Baixa a planilha analítica da matriz 4Ps (`relatorio_4ps.xlsx`) contendo abas separadas para "Pareceres" e "Notas 4Ps".
    - **Cabeçalho de Resposta:** `Content-Disposition: attachment; filename="relatorio_4ps.xlsx"`

- **Resposta:** `200 OK` (`application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`) ou `409 Conflict`.

(Nota: Os relatórios quantitativos FAIR estendidos — `relatorio_fair.html` e `relatorio_fair.xlsx` — são gerados em disco via `analysis_service.py` durante execuções de simulação estocástica offline).

A sustentação desses endpoints em ambiente de produção exige mecanismos rigorosos de validação, controle de concorrência e segurança.

6. Requisitos Não Funcionais, Concorrência e Segurança

Os requisitos não funcionais determinam as salvaguardas operacionais que garantem a resiliência do sistema, a sanidade dos dados e o isolamento completo das inferências de Inteligência Artificial.

6.1. Segurança da Informação, Privacidade e Isolamento de Loopback

Para assegurar a operação em ambiente totalmente isolado (*air-gapped*) e impedir a exfiltração não autorizada de dados corporativos ou ataques de *proxy-jacking*, o arquivo `local_ai.py` implementa uma verificação rigorosa de URL através da função `_validate_local_url()`:

```
def _validate_local_url() -> None:
    hostname = urlparse(OLLAMA_BASE_URL).hostname
    if hostname not in {"127.0.0.1", "localhost", "::1"}:
        raise RuntimeError("A IA deve usar um servidor Ollama local neste computador.")
```

Essa função extrai o *hostname* do endereço configurado em `OLLAMA_BASE_URL` e o valida estritamente contra o conjunto de endereços de *loopback* local (`127.0.0.1`, `localhost`, `::1`). Se o endereço apontar para uma rede externa ou IP público, o sistema lança um `RuntimeError` imediatamente, interrompendo a requisição.

Além disso, a criação do cliente HTTP no módulo `local_ai.py` força a desativação de proxies do sistema operacional utilizando `build_opener(ProxyHandler({}))`. Essa medida neutraliza variáveis de ambiente globais como `HTTP_PROXY` ou `HTTPS_PROXY`, garantindo zero vazamento de dados em enclaves corporativos restritos.

6.2. Integridade dos Dados e Validação de Esquema com Pydantic

A validação das requisições REST é realizada na camada de entrada utilizando esquemas do **Pydantic v2** declarados no arquivo `api.py`.

Modelo Pydantic	Regras de Validação e Limites de Caracteres	Validações Customizadas (`@field_validator` / `@model_validator`)
`RiskDimensionInput`	`dimension`: Lista restrita dos 4Ps. `severity` / `probability`: Inteiros entre 1 e 5. `evidence` / `justification`: Mínimo 10 e máximo 1200 caracteres.	`strip_dimension_text`: Aplica `.strip()` e rejeita textos com menos de 10 caracteres úteis.
`DimensionExplanationRequest`	`title`: 1 a 200 caracteres. `category`: 1 a 100 caracteres. `description`: Opcional (até 2000 caracteres). `evidence`: Mínimo 10 e máximo 1200 caracteres.	`strip_required_text`: Garante que os campos obrigatórios não contenham apenas espaços em branco.
`RiskPreventionDimension`	`dimension`: Lista restrita dos 4Ps. `severity` / `probability`: Inteiros de 1 a 5. `evidence` / `justification`: Opcionais (padrão `""`, até 1200 caracteres, sem mínimo de 10).	Utilizado internamente no payload de requisição de recomendações preventivas.
`RiskPreventionRequest`	`title` (1–200), `category` (1–100), `description` (opcional). Lista `dimensions`	`validate_all_dimensions`: Valida a presença exata de todas as 4 dimensões sem duplicações ou omissões.

	obrigatoriamente com 4 itens.	
RiskCreate	`title` (1–200), `category` (1–100), `description` (opcional). Lista `dimensions` de 4 itens (`RiskDimensionInput`).	`validate_all_dimensions`: Garante a obrigatoriedade do preenchimento estrito dos 4Ps com validação de 10 a 1200 caracteres.
RiskDimensionsUpdate	Lista `dimensions` com exatamente 4 itens (`RiskDimensionInput`).	Invoca `validate_dimensions` para garantir a presença e sanidade das 4 dimensões.

6.3. Concorrência, Threading Locks e Proteção contra Saturação de CPU

A execução da análise matricial e das simulações estatísticas pode demandar uso intensivo de CPU. Para evitar condições de corrida (*race conditions*) e proteger os recursos do servidor contra saturação por requisições concorrentes, o arquivo `api.py` emprega uma trava de exclusão mútua (`Lock`) da biblioteca nativa `threading`:

```
_analysis_progress_lock = Lock()
_analysis_active = False
_analysis_progress: dict[str, Any] = {
    "status": "idle",
    "percent": 0,
    "message": "Nenhuma análise em andamento.",
}
```

Ao receber uma chamada no endpoint `POST /api/analysis`, a aplicação adquire a trava `_analysis_progress_lock`. Se a *flag* `_analysis_active` for `True`, a requisição é rejeitada imediatamente retornando o código HTTP `409 Conflict` com a mensagem `"Já existe uma análise em andamento."`. Caso contrário, o estado é alterado para ativo, garantindo atualizações *thread-safe* no dicionário global de progresso (`_analysis_progress`) durante toda a execução e liberando a trava com segurança ao término do processamento.

A arquitetura do 'Risco em Perspectiva' entrega uma solução sólida, modular e totalmente autônoma para a gestão e quantificação de riscos operacionais. A combinação de uma interface reativa em Vue 3, uma API robusta em FastAPI baseada em um padrão de duplo motor analítico, um banco SQLite auto-migrável, simulações estatísticas Monte Carlo (OpenFAIR) e inferência de linguagem 100% offline via Ollama assegura que a organização obtenha análises de alto valor analítico sob total conformidade com as diretrizes de segurança e privacidade de dados.